Reviewer Pack — SOS Degree Lower Bound via Matroid Polytope Dimension

Entry b4a7c853b857 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 16:28:41 UTC

SOS Degree Lower Bound via Matroid Polytope Dimension

Entry ID: `b4a7c853b857` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 16:28:41 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Matroid Polytopes) **Field B** (complexity object): SOS Refutation Degree for CSPs

Statement:

> For a CSP instance represented as a hypergraph H, the SOS refutation degree required to prove unsatisfiability is at least the dimension of the matroid polytope formed by the characteristic vectors of H's hyperedges. Formally, dim(polytope(H)) ≤ deg_SOS(H) for all hypergraphs H with n ≤ 40.

Rationale (proposer's reasoning):

> Matroid polytopes encode combinatorial constraints via convex hulls, while SOS degrees measure algebraic complexity of CSPs. Their direct linkage could reveal geometric obstructions to polynomial-time refutations, leveraging the interplay between discrete geometry and algebraic optimization.

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `73ce9fdc25432a41`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=247.5s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    m = 3 * n

    # Generate a random 3-CNF instance
    clauses = []
    for _ in range(m):
        variables = [random.randint(1, n) for _ in range(3)]
        clause = tuple(sorted(variables))
        if random.choice([True, False]):
            clause = (-clause[0], -clause[1], -clause[2])
        clauses.append(clause)

    # Construct the matroid polytope
    hyperedges = set()
    for clause in clauses:
        hyperedges.add(frozenset(clause))

    characteristic_vectors = []
    for i in range(2**n):
        vector = [0] * n
        for j in range(n):
            if (i >> j) & 1:
                vector[j] = 1
        characteristic_vectors.append(vector)

    # Compute the dimension of the matroid polytope using Gaussian elimination
    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for j in range(n):
            pivot_row = -1
            for i in range(rank, m):
                if A[i][j] != 0:
                    pivot_row = i
```

```
                    break
            if pivot_row == -1:
                continue
            A[pivot_row], A[rank] = A[rank], A[pivot_row]
            for k in range(n):
                if k != j and A[rank][k] != 0:
                    factor = A[rank][k] / A[rank][j]
                    for l in range(n):
                        A[rank][l] -= factor * A[pivot_row][l]
            rank += 1
        return rank

    matroid_polytope_dimension = gaussian_elimination(characteristic_vectors)

    # Compute the minimal SOS degree required to refute the instance
    def is_satisfiable(clauses):
        assignment = [random.choice([-1, 1]) for _ in range(n)]
        for clause in clauses:
            if all(assignment[var-1] * literal >= 0 for var, literal in enumerate(clause)):
                return True
        return False

    def sos_degree(clauses):
        degree = 0
        while not is_satisfiable(clauses):
            degree += 1
            # Add a new SOS constraint to refute the instance
            # This is a simplified version and may not always work
            for i in range(n):
                clauses.append((i+1, -i-1))
        return degree

    sos_refutation_degree = sos_degree(clauses)

    # Verify the conjecture
    conjecture_holds = matroid_polytope_dimension <= sos_refutation_degree
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "SOS Degree Lower Bound",
        "metric_value": sos_refutation_degree,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]]
    if not seeds:
        seeds = [2**i + 1 for i in range(5, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
```

```
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractior
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	110887
2	propose	ollama_remote	qwen3:8b	0	0	86992
3	preregistration	ollama_remote	qwen3:8b	0	0	26186
4	novelty	ollama_remote	qwen3:8b	0	0	40975
5	novelty	ollama_remote	qwen3:8b	0	0	17320
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23362
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11119
8	judge	ollama_remote	qwen3:8b	0	0	110565

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 427405 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/b4a7c853b857.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b4a7c853b857.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b4a7c853b857.tar.gz` (if generated)